Android Data Binding: Write Less to do More

The Android Data Binding framework leads to a significant reduction in code and, as a bonus, comes with other goodies such as BindingAdapters and Auto-Magic attributes, at almost no performance cost.

Let's face it, we all love Android, but we don't really like one of the side-effects of Android development, which is *boilerplate code*. JAVA is a verbose language, and when it comes to working with a UI framework, the code that is written to essentially glue your UI to your model is really painful. The code to glue the UI to your model is important, but is something that can be eliminated with technology. The simple steps for gluing the UI to the model in Android can be as follows:

- First, get a reference to the view with *findViewById()*
- Next, load the model from the disc or over the network
- Then load the data from the model into the view; for example, setting text in TextView or loading an image in ImageView
- Now, if any changes happen in the UI which affect the model, update the model
- And if the changed value is displayed elsewhere in the UI, update those views too

Now, this isn't the only way we can bind the model to the UI, but do realise that we're writing fair amounts of code just to get our data to the view.

Data binding

Data binding, in general: A typical definition from Wikipedia states that data binding is a general technique that binds data sources from the provider and consumer together, and synchronises them. What this means descriptively is that data binding is a technique in which the UI and the model are connected in such a way that the UI as well as the model generate proper notifications about data changes, and use those notifications to keep each other updated about data.

Types of data binding: There are typically two types of data binding. The first is one-way data binding and the other is two-way data binding. These operate pretty much as their names suggest. One-way data binding is when data flows in only one direction, which is, from the model or the data to the UI. An example of it can be when a TextView sets its data from a variable and keeps the text of the view updated with the variable whenever the variable changes.

With two-way data binding, the data flows in both directions, i.e., from the model to the UI and from the UI to the model. An example would be TextView and EditText — the TextView shows the contents of EditText. Now, if I bind EditText and TextView with the model in two-way binding, when we write some text in EditText, the data of EditText is passed to the model or variable, which is the flow from UI to model. When the variable is updated, it is bound with TextView, so the contents of TextView are also updated, and this flow is from the model or the variable to the UI. This is how data flows in two ways; hence, the name two-way data binding. It'll all make more sense when we get to the code.

Writing the first app

First, you need to make your project data binding-enabled. You do that by adding the following code to the *build.gradle* file for the module of your app, and not to the global *build.gradle* file for the whole project. Note that this is the way to enable data binding in Studio 2.0 and later. You can search on the Internet about enabling it in versions below 2.0.

```
android {
 ....
 dataBinding {
 enabled = true
 }
}
```

Now, let's consider a simple model named User. It contains two fields — the first name and last name.